CSE222 HW8 REPORT

MUHAMMET FURKAN YILGIN
220104004964

IMPLEMENTATIONS

Adding Person

```java
/**
 * Method to add person to people map.First the inputs of name , age and hobbies are taken in the Main class main method then person is created
 * with these datas after that this person is added to people map and added to the friendship graph with an empty list for friends.
 * I did not include timestamp while taking the input since your code also did not include it and also it is hard to enter the exact date from keyboard.
 * @param Person_Name
 * @param age
 * @param hobbies
 */
public void AddPerson(String Person_Name , int age , String hobbies ){
    Person newPerson = new Person(Person_Name, age, hobbies);
    People.put(Person_Name, newPerson);
    Friendships.put(newPerson, new ArrayList<>());
    System.out.println("Person added -> " + newPerson) ;
}
```

Removing Person

```java
/**
 * Method to remove person from people map.First the input of name is taken in the Main class main method then person matches
 * with this name is removed and also it is removed from friendsip graph too both key and values.
 * An error message will be printed if the person to be deleted does not exist.
 * @param Person_Name
 * @param age
 * @param hobbies
 */
public void RemovePerson(String Person_Name){
    Person deletePerson = People.get(Person_Name);
    if(deletePerson!=null){
        People.remove(deletePerson); // removing from people map
        Friendships.remove(deletePerson); // removing from friendship graph key.
        for (Person p : Friendships.keySet()) { // removing from other person's friendship list
            List<Person> friends = Friendships.get(p);
            for(int i=0 ; i<friends.size() ; i++){
                if(friends.get(i).equals(deletePerson))
                    friends.remove(i);
            }
        }

        System.out.println("Person " + deletePerson + " IS DELETED !");
    }
    else {
        System.out.println("The person you are trying to remove does not exist!");
    }
}
```

Adding Friendship

```java
/**
 * Method to add friendship between two person.It gets the 2 person from their names then if both of these persons exist
 * it adds friendship between them for both of them one by one.
 * @param person1_name
 * @param person2_name
 */
public void AddFriendship(String person1_name , String person2_name){

    Person person1 = People.get(person1_name);
    Person person2 = People.get(person2_name);

    if (person1 != null && person2 != null) {
        Friendships.get(person1).add(person2);
        Friendships.get(person2).add(person1);
        System.out.println("Friendship added between " + person1_name + " and " + person2_name);
    } else {
        System.out.println("Friendship can not be added ! One or both persons not found in the network.");
    }
}
```

Removing Friendship

```java
/**
 * Method to remove friendship between two person.It gets the 2 person from their names then if both of these persons exist
 * it removes the friendship between them for both of them one by one.
 * @param person1_name
 * @param person2_name
 */
public void RemoveFriendship(String person1_name , String person2_name){

    Person person1 = People.get(person1_name);
    Person person2 = People.get(person2_name);

    if (person1 != null && person2 != null) {
        Friendships.get(person1).remove(person2);
        Friendships.get(person2).remove(person1);
        System.out.println("Friendship removed between " + person1_name + " and " + person2_name);
    } else {
        System.out.println("Friendship can not be deleted! One or both persons not found in the network.");
    }

}
```

Suggesting Friends

```java
/**
 * method to suggest friends for the given person and given number of suggestions.
 * First the person with that name is found.If there is no such person or suggestion number is < 1 then error message will be printed.
 * Otherwise the hobbies will be checked first then the hobby count will be calculated
 * after that mutual friends will be calculated and according to that for all person objects and score will be calculated and then scores will be sorted
 * and first N(suggestion number) of this sorted persons will be printed.
 * @param name
 * @param suggestion_number
 */
public void SuggestFriends(String name, int suggestion_number) {
    // Check if the person exists and suggestion number is valid
    if (People.containsKey(name) && suggestion_number > 0) {
        System.out.println("Suggested friends for " + name + ":");

        // List to store potential friends and their scores
        List<Person> suggestions = new ArrayList<>();

        // Map to store total scores and mutual friend counts for each potential friend
        Map<Person, Double> scores = new HashMap<>();
        Map<Person, Integer> mutualFriendCounts = new HashMap<>();

        // Iterate over all persons in the graph
        for (Person p : People.values()) {
            // Skip if it is the same person or they are already friends(contains)
            if (!p.getName().equals(name) && !Friendships.getOrDefault(People.get(name), new ArrayList<>()).contains(p)) {
                // Counter for common hobbies
                double hobby_count = 0;

                // Calculating common hobbies
                for (String hobby : p.getHobbies()) {
                    if (People.get(name).getHobbies().contains(hobby)) {
                        hobby_count += 0.5;
                    }
                }
```

```java
                // Calculating mutual friend count
                int mutualFriendCount = 0;
                for (Person friend : Friendships.get(p)) {
                    if (Friendships.get(People.get(name)).contains(friend)) {
                        mutualFriendCount++;
                    }
                }

                // Calculating total score
                double total = hobby_count + mutualFriendCount;

                // Store the total score and mutual friend count for the potential friend
                scores.put(p, total);
                mutualFriendCounts.put(p, mutualFriendCount);
            }
        }

        // Sort the potential friends based on the total score
        List<Map.Entry<Person, Double>> sortedScores = new ArrayList<>(scores.entrySet());
        sortedScores.sort(Map.Entry.comparingByValue(Comparator.reverseOrder()));

        // Print the top suggestion_number suggestions
        for (int i = 0; i < Math.min(suggestion_number, sortedScores.size()); i++) {
            Map.Entry<Person, Double> entry = sortedScores.get(i);
            Person suggestedFriend = entry.getKey();
            double total = entry.getValue();
            int mutualFriendCount = mutualFriendCounts.get(suggestedFriend);
            System.out.println(suggestedFriend.getName() + " ( Score: " + total + ", " + mutualFriendCount + " mutual friends, " + (total-mutualFriendCount) / 0.5
            suggestions.add(suggestedFriend);
        }

        // If not enough suggestions were found
        if (suggestions.size() < suggestion_number) {
            System.out.println("Not enough suggestions available.");
        }
```

Finding the Shortest Path

```java
/**
 * Finds and prints the shortest path between two persons in the friendship graph using BFS.
 * @param startName The name of the starting person.
 * @param endName The name of the ending person.
 */
public void FindShortestPath(String startName, String endName) {
    // Getting the Person objects for the given names
    Person start = People.get(startName);
    Person end = People.get(endName);

    // Check if both persons exist in the graph
    if (start == null || end == null) {
        System.out.println("One or both persons not found in the graph.");
        return;
    }

    // Initialize a queue for BFS, a map to keep track of previous nodes, and a set for visited nodes
    Queue<Person> queue = new LinkedList<>();
    Map<Person, Person> prev = new HashMap<>();
    Set<Person> visited = new HashSet<>();

    // Start BFS from the starting person
    queue.add(start);
    visited.add(start);

    // Perform BFS
    while (!queue.isEmpty()) {
        Person current = queue.poll();

        // If the current person is the end person then print the path and return
        if (current.equals(end)) {
            printPath(start, end, prev);
            return;
        }

        // Iterate over the neighbors of the current person
        for (Person neighbor : Friendships.getOrDefault(current, new ArrayList<>())) {
            // If the neighbor has not been visited, adding it to the queue and mark it as visited
            if (!visited.contains(neighbor)) {
                queue.add(neighbor);
                visited.add(neighbor);
                // Keeping track of the path by storing the previous node for each neighbor
                prev.put(neighbor, current);
            }
        }
    }

    // If the queue is empty and the end person was not found, print that no path was found
    System.out.println("No path found between " + startName + " and " + endName);
}
```

Counting Clusters

```java
/**
 * Counts and displays the clusters in the friendship graph.
 * @return the count of clusters.
 */
public int CountClusters() {
    // Set to keep track of visited persons
    Set<Person> visited = new HashSet<>();
    // Variable to count the number of clusters
    int count = 0;
    // List to store all clusters
    List<List<Person>> clusters = new ArrayList<>();

    // Iterate over all persons in the friendship graph
    for (Person person : Friendships.keySet()) {
        // If the person is not visited  perform bfs to find the cluster
        if (!visited.contains(person)) {
            // List to store the current cluster
            List<Person> cluster = new ArrayList<>();
            // Performing bfs starting from the current person to find all connected persons
            bfs(person, visited, cluster);
            // Add the found cluster to the list of clusters
            clusters.add(cluster);
            // Incrementing the cluster count
            count++;
        }
    }

    // Variable to keep track of the cluster index for display purposes
    int clusterIndex = 1;
    // Iterate over all clusters to print their members
    for (List<Person> cluster : clusters) {
        System.out.println("Cluster " + clusterIndex + ":");
        for (Person person : cluster) {
            // Print each person in the current cluster
            System.out.println(person);
        }
        // Increment the cluster index
        clusterIndex++;
    }

    // Return the total count of clusters
    return count;
}
```

Main Method (The Menu)

- Here is some parts of the main method.

- As I stated in comments of their methods.I did not take the timestamp input from the user since it was not included in the provided code you have shared with us and also it was hard to enter the exact time correctly from keyboard too.

```
Scanner scanner = new Scanner(System.in);
int choice = 0;
SocialNetworkGraph sng = new SocialNetworkGraph();

while (choice != 8) {
    System.out.println("===== Social Network Analysis Menu =====");
    System.out.println("1. Add person");
    System.out.println("2. Remove person");
    System.out.println("3. Add friendship");
    System.out.println("4. Remove friendship");
    System.out.println("5. Find shortest path");
    System.out.println("6. Suggest friends");
    System.out.println("7. Count clusters");
    System.out.println("8. Exit");
    System.out.print("Please select an option: ");
    try{
        if (scanner.hasNextInt()) {
            choice = scanner.nextInt();
            scanner.nextLine();
            String timestamp1 , timestamp2 ;
            switch (choice) {
                case 1:
                    System.out.print("Enter name : ");
                    String name = scanner.nextLine() ;
                    System.out.print("Enter age : ");
                    int age  = scanner.nextInt() ;
                    scanner.nextLine();
                    System.out.print("Enter hobbies (comma-separated): ");
                    String hobbies = scanner.nextLine() ;
                    sng.AddPerson(name , age , hobbies );
                    break;
```

```
                case 2:
                    System.out.print("Enter name of the person to be deleted : ");
                    name = scanner.nextLine() ;
                    sng.RemovePerson(name);
                    break;
                case 3:
                    System.out.print("Enter first person's name : " );
                    name = scanner.nextLine() ;
                    /*System.out.print("Enter first person's timestamp: " );
                    timestamp1 = scanner.nextLine();*/
                    System.out.print("Enter second person's name: ");
                    String name2 = scanner.nextLine() ;
                    /*System.out.print("Enter second person's timestamp: ");
                    timestamp2 = scanner.nextLine();*/
                    sng.AddFriendship(name , name2 );
                    break;
                case 4:
                    System.out.print("Enter first person's name : " );
                    name = scanner.nextLine() ;
                    /*System.out.print("Enter first person's timestamp: " );
                    timestamp1 = scanner.nextLine();*/
                    System.out.print("Enter second person's name: ");
                    name2 = scanner.nextLine() ;
                    /*System.out.print("Enter second person's timestamp: ");
                    timestamp2 = scanner.nextLine();*/
                    sng.RemoveFriendship( name , name2 );
                    break;
```

EXAMPLE OUTPUTS

From demonstration examples

```
public static void main(String[] args){

    SocialNetworkGraph network = new SocialNetworkGraph();
    // Adding some people for demonstration

    network.AddPerson("John Doe", 25, "reading,hiking,cooking");
    network.AddPerson("Jane Smith", 22, "swimming,cooking");
    network.AddPerson("Alice Johnson", 27, "hiking,painting");
    network.AddPerson("Bob Brown", 30, "reading,swimming");
    network.AddPerson("Emily Davis", 28, "running,swimming");
    network.AddPerson("Frank Wilson", 26,"reading,hiking");


    // Adding friendships for demonstration
    network.AddFriendship("John Doe", "Jane Smith");
    network.AddFriendship("John Doe", "Alice Johnson");
    network.AddFriendship("Jane Smith", "Bob Brown");
    network.AddFriendship("Emily Davis", "Frank Wilson");
    //network.AddFriendship("Bob Brown","Emily Davis");

    network.SuggestFriends("John Doe",2);
    // Finding shortest path for demonstration
    //network.FindShortestPath("John Doe","Emily Davis");
    network.FindShortestPath("John Doe","Bob Brown");


    // Counting clusters for demonstration
    network.CountClusters();

    System.out.println("DISPLAYING FRIENDSHIPS");
    network.displayFriendships();
   network.RemovePerson("Emily Davis");
   network.RemoveFriendship("John Doe","Jane Smith");

   System.out.println("DISPLAYING FRIENDSHIPS");
   network.displayFriendships();

   network.FindShortestPath("John Doe","Bob Brown");
```

Outputs are :

```
mfurkan@DESKTOP-625529P:/mnt/c/Users/mfurk/Desktop/VSC Codes/CSE222-HW8$ make
rm -f *.class
javac -d . Person.java SocialNetworkGraph.java Main.java
java Main
Person added -> Name : John Doe Age : 25 Hobbies = [reading, hiking, cooking] Creation Time = Wed May 29 22:28:57 TRT 2024
Person added -> Name : Jane Smith Age : 22 Hobbies = [swimming, cooking] Creation Time = Wed May 29 22:28:57 TRT 2024
Person added -> Name : Alice Johnson Age : 27 Hobbies = [hiking, painting] Creation Time = Wed May 29 22:28:57 TRT 2024
Person added -> Name : Bob Brown Age : 30 Hobbies = [reading, swimming] Creation Time = Wed May 29 22:28:57 TRT 2024
Person added -> Name : Emily Davis Age : 28 Hobbies = [running, swimming] Creation Time = Wed May 29 22:28:57 TRT 2024
Person added -> Name : Frank Wilson Age : 26 Hobbies = [reading, hiking] Creation Time = Wed May 29 22:28:57 TRT 2024
Friendship added between John Doe and Jane Smith
Friendship added between John Doe and Alice Johnson
Friendship added between Jane Smith and Bob Brown
Friendship added between Emily Davis and Frank Wilson
Suggested friends for John Doe:
Bob Brown ( Score: 1.5, 1 mutual friends, 1.0 common hobbies)
Frank Wilson ( Score: 1.0, 0 mutual friends, 2.0 common hobbies)
Shortest path : John Doe -> Jane Smith -> Bob Brown
Cluster 1:
Name : Frank Wilson Age : 26 Hobbies = [reading, hiking] Creation Time = Wed May 29 22:28:57 TRT 2024
Name : Emily Davis Age : 28 Hobbies = [running, swimming] Creation Time = Wed May 29 22:28:57 TRT 2024
Cluster 2:
Name : John Doe Age : 25 Hobbies = [reading, hiking, cooking] Creation Time = Wed May 29 22:28:57 TRT 2024
Name : Jane Smith Age : 22 Hobbies = [swimming, cooking] Creation Time = Wed May 29 22:28:57 TRT 2024
Name : Alice Johnson Age : 27 Hobbies = [hiking, painting] Creation Time = Wed May 29 22:28:57 TRT 2024
Name : Bob Brown Age : 30 Hobbies = [reading, swimming] Creation Time = Wed May 29 22:28:57 TRT 2024
DISPLAYING FRIENDSHIPS
Frank Wilson is friends with: Emily Davis
John Doe is friends with: Jane Smith, Alice Johnson
Emily Davis is friends with: Frank Wilson
Jane Smith is friends with: John Doe, Bob Brown
Bob Brown is friends with: Jane Smith
Alice Johnson is friends with: John Doe
Person Name : Emily Davis Age : 28 Hobbies = [running, swimming] Creation Time = Wed May 29 22:28:57 TRT 2024 IS DELETED !
Friendship removed between John Doe and Jane Smith
Alice Johnson is friends with: John Doe
Person Name : Emily Davis Age : 28 Hobbies = [running, swimming] Creation Time = Wed May 29 22:28:57 TRT 2024 IS DELETED !
Friendship removed between John Doe and Jane Smith
DISPLAYING FRIENDSHIPS
Frank Wilson is friends with:
John Doe is friends with: Alice Johnson
Jane Smith is friends with: Bob Brown
Bob Brown is friends with: Jane Smith
Alice Johnson is friends with: John Doe
No path found between John Doe and Bob Brown
===== Social Network Analysis Menu =====
1. Add person
2. Remove person
3. Add friendship
4. Remove friendship
5. Find shortest path
6. Suggest friends
7. Count clusters
8. Exit
Please select an option: 8
Exiting...
```

From menu

```
Please select an option: 1
Enter name : Furkan
Enter age : 22
Enter hobbies (comma-separated): football
Person added -> Name : Furkan Age : 22 Hobbies = [football] Creation Time = Wed May 29 22:36:38 TRT 2024
===== Social Network Analysis Menu =====
1. Add person
2. Remove person
3. Add friendship
4. Remove friendship
5. Find shortest path
6. Suggest friends
7. Count clusters
8. Exit
Please select an option: 1
Enter name : Muhammet
Enter age : 20
Enter hobbies (comma-separated): football,f1
Person added -> Name : Muhammet Age : 20 Hobbies = [football, f1] Creation Time = Wed May 29 22:36:50 TRT 2024
===== Social Network Analysis Menu =====
1. Add person
2. Remove person
3. Add friendship
4. Remove friendship
5. Find shortest path
6. Suggest friends
7. Count clusters
8. Exit
Please select an option: 6
Enter person's name : Furkan
Enter maximum number of friends to suggest: 1
Suggested friends for Furkan:
Muhammet ( Score: 0.5, 0 mutual friends, 1.0 common hobbies)
```

:

```
4. Remove friendship
5. Find shortest path
6. Suggest friends
7. Count clusters
8. Exit
Please select an option: 7
Counting clusters in the social network...
Cluster 1:
Name : Muhammet Age : 20 Hobbies = [football, f1] Creation Time = Wed May 29 22:36:50 TRT 2024
Cluster 2:
Name : Furkan Age : 22 Hobbies = [football] Creation Time = Wed May 29 22:36:38 TRT 2024
Number of clusters found : 2
===== Social Network Analysis Menu =====
1. Add person
2. Remove person
3. Add friendship
4. Remove friendship
5. Find shortest path
6. Suggest friends
7. Count clusters
8. Exit
Please select an option: 5
Enter first person's name : Muhammet
Enter second person's name: Furkan
No path found between Muhammet and Furkan
===== Social Network Analysis Menu =====
1. Add person
2. Remove person
3. Add friendship
4. Remove friendship
5. Find shortest path
6. Suggest friends
7. Count clusters
8. Exit
Please select an option: 8
Exiting...
```